
يَسْمِ اللّٰهَ الرَّحْمٰنَ الرَّحِيْمَ



Natural language Processing

Lecture – Feature Extraction from Text

Author: Dr. Rao Muhammad Adeel Nawab

Instructor: Muhammad Ehsan

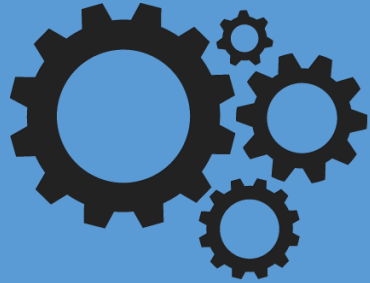
Lecture Outline

- **Features in NLP / Machine Learning**
- **Representing Textual Features as N-grams**
- **Feature Extraction Examples**
- **Feature Extraction + Feature Selection Examples**

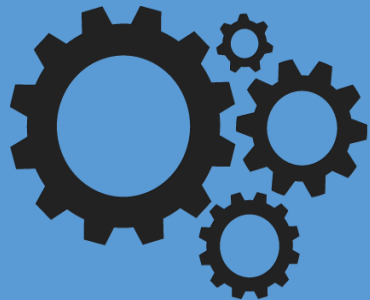


Features in NLP/ Machine Learning

What is a Feature?



A feature (Attribute) is an individual property or characteristic used as input to a machine learning model.



Many algorithms in machine learning require a numerical representation, since such representations facilitate processing and statistical analysis

👉 Raw Text → Preprocessing → **Feature Extraction** → Model Selection & Training → Prediction → Deployment

Feature Extraction

- Feature Extraction Is the process of extracting “features” (or attributes) from data in a format supported by machine learning algorithms
- Data can be
 - Text
 - Image
 - Video
 - Audio
- Scikit-Learn module for feature extraction
 - `sklearn.feature_extraction`

Feature Selection

- Choosing informative, discriminating and independent features is a crucial step in machine learning
- Feature Selection
 - is the process of selecting a “subset of relevant features” from the original set of extracted features
- Goal of Feature Selection is to improve machine learning algorithms' Accuracy or to boost their performance on very high-dimensional data
- Scikit-Learn module for feature selection
 - `sklearn.feature_selection`

Feature Vector



Definition

- ▮ A feature vector is just a vector that contains information describing an object's important characteristics



More Formal Definition

- ▮ A feature vector is an n-dimensional vector of numerical features that represents some object



In Text Data

- ▮ Feature = Words / Terms / Tokens

Feature Vector



In Text Data

- Feature = Frequency Counts of Words in a Text



Example - Feature Vector of a Text

- A vector “frequency count” of each word (or feature) appearing in the text

Sample Text: a nice car car

Text Features = <a, nice, car>

Text Feature Vector = <1, 1, 2>

Vocabulary and Stop Words



Vocabulary

□ The set of unique words (or tokens) in a dataset / corpus



Example

Sentences:

“I like NLP” , “I like AI”



Vocabulary:

[I, like, NLP, AI]

✓ Duplicate words are **not repeated**

✓ Only **unique words** are included



Stop Words

□ The set of commonly used words

Vocabulary and Stop Words



Example – Consider the following corpus

Comment	class
a nice car	Positive
a good car	Positive
a beautiful car a nice car	Positive
a bad black car car	Negative



Vocabulary (with stop words)

□ {a, nice, car, good, beautiful, bad, black}



Vocabulary (without stop words)

□ {nice, car, good, beautiful, bad, black}

Feature Representation of Textual Data



In Text

- ▮ Features are made up of words (or tokens) / characters



Transforming String (Text) to Vector

- ▮ Transform documents (or corpus or collection of texts) to document-term matrix

Feature Representation of Textual Data (cont...)

Method 1: Bag of Words (BoW) / Count Vectorizer

Two distinct Weighting parameters of Bag of Words (BoW):

- **Binary Weight**

Assign 1 if a feature occurs in a text document otherwise 0

- **Count or (Term Frequency Count) Weight**

Use “frequency count” of a feature as weight, It is the raw number of times a word appears in a document.

Feature Representation of Textual Data (cont...)

Method 2: Term Frequency (TF) - Inverse Document Frequency (IDF) (TFIDF)

○ Term Frequency (TF) Weight

Term Frequency (TF) tells how common or important a word is in a document

Measures how often a word appears / Importance **within one document**

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

○ Inverse Document Frequency (IDF)

IDF measures how important a word is across all documents.

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}} \right)$$

○ TF.IDF Weight

$$TF.IDF(t) = TF(t) * IDF(t)$$



Representing Textual Features as N-grams

N-gram Models

- An n-gram is a contiguous sequence of n items from a given sample of text
- N-gram can be
 - Word based
 - Character based
- N represents the length of N-gram

Example – N-gram Generation from Input Text



Input Text

▮ R u coming?



Word Uni-grams (N = 1) -> (Min, Max) (1,1)

▮ Tokenized Text:

R

u

coming

?

▮ Set of Word Uni-grams = {R, u, coming,?}

Example – N-gram Generation from Input Text



Input Text

▮ R u coming?



Word Bi-grams (N = 2) -> (2,2)

▮ Tokenized Text:

R

u

coming

?

▮ Set of Word Bi-grams = {R u, u coming, coming ?}

Example – N-gram Generation from Input Text



Input Text

▮ R u coming?



Word Tri-grams (N = 3) -> (3,3)

▮ Tokenized Text:

R

u

coming

?

▮ Set of Word Tri-grams = {R u coming, u coming ? }

Example – N-gram Generation from Input Text



Input Text

□ R u coming?



Word Uni-grams + Bigram (N = 1) -> (Min, Max) (1,2)

□ Tokenized Text:

R

u

coming

?

□ Set of Word Uni-grams = {R, u, coming, ?, R u, u coming, coming ?}

Example – N-gram Generation from Input Text



Input Text

□ R u coming?



Character Tri-grams (N = 3)

□ Tokenized Text: Note that **space** is also a character

R		u		C	o	m	i	n	g	?
---	--	---	--	---	---	---	---	---	---	---

□ Set of Character Tri-grams = {R u, u ,u c, co,com,omi,min,ing,ng?}



Feature Extraction Examples

Example 01 – Representing Textual Features as N-grams



Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive



N-gram = word

- Min length of N = 1
- Max length of N = 1



Stop Words Removal = False



Max Features = Vocabulary Size



Feature Extraction Method = Bow with Binary Weight

Example 01 – Representing Textual Features as N-grams



Document-Term Matrix (Binary Weights)

Vocabulary:

[a, nice, car, good, beautiful, black]

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

A	Nice	Car	Good	Beautiful	Black
1	1	1	0	0	0
1	0	1	1	0	0
1	1	1	0	1	0
1	1	1	0	0	1

Example 02 – Representing Textual Features as N-grams

 N-gram = word

-  Min length of N = 1
-  Max length of N = 1

 Stop Words Removal = False

 Max Features = Vocabulary Size

 Feature Extraction Method = Count or (Term Frequency Count) Weight

 Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 02 – Representing Textual Features as N-grams



Document-Term Matrix: Count or (Term Frequency Count) Weight

A	Nice	Car	Good	Beautiful	Black	Class
1	1	1	0	0	0	Pos
1	0	1	1	0	0	Pos
2	1	2	0	1	0	Pos
1	1	2	0	0	1	Pos

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 03 – Representing Textual Features as N-grams



N-gram = word

- Min length of N = 1
- Max length of N = 1



Stop Words Removal = False



Max Features = Vocabulary Size



Feature Method = (TFIDF)



Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}}\right)$$

Example 03 – Representing Textual Features as N-grams

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

Document 1: "a nice car" (3 words)

- $TF("a") = 1/3$
- $TF("nice") = 1/3$
- $TF("car") = 1/3$

Document 2: "a good car" (3 words)

- $TF("a") = 1/3$
- $TF("good") = 1/3$
- $TF("car") = 1/3$

Document 3: "a beautiful car a nice car" (6 words)

- $TF("a") = 2/6 = 1/3$
- $TF("beautiful") = 1/6$
- $TF("car") = 2/6 = 1/3$
- $TF("nice") = 1/6$

Document 4: "a nice black car car" (5 words)

- $TF("a") = 1/5$
- $TF("nice") = 1/5$
- $TF("black") = 1/5$
- $TF("car") = 2/5$

IDF Calculations:

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}}\right)$$

- $IDF("a") = \log(4/4) = 0$
- $IDF("nice") \approx \log(4/3) \approx 0.12494$
- $IDF("car") = \log(4/4) = 0$
- $IDF("good") \approx \log(4/1) \approx 0.60206$
- $IDF("beautiful") \approx \log(4/1) \approx 0.60206$
- $IDF("black") \approx \log(4/1) \approx 0.60206$



Example 03 – Representing Textual Features as N-grams



Document-Term Matrix (TF-IDF Weights)

A	Nice	Car	Good	Beautiful	Black
0	0.0416	0	0	0	0
0	0	0	0.20069	0	0
0	0.0208	0	0	0.100	0
0	0.0249	0	0	0	0.1204

Example 04 – Representing Textual Features as N-grams



N-gram = word

- Min length of N = 1
- Max length of N = 1



Stop Words Removal = True



Max Features = Vocabulary Size



Feature Method = Count or (Term Frequency Count) Weight



Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 04 – Representing Textual Features as N-grams



Document-Term Matrix (with Stop Words Removal)

Nice	Car	Good	Beautiful	Black
1	1	0	0	0
0	0	1	0	0
1	2	0	1	0
1	2	0	0	1



Feature Extraction + Feature Selection Examples

Example 01 – Representing Textual Features as N-grams

 N-gram = word

- ▮ Min length of N = 1
- ▮ Max length of N = 1

 Stop Words Removal = True

 Max Features = 4

 Feature Method = Count or (Term Frequency Count) Weight

 Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 01 – Representing Textual Features as N-grams



Document-Term Matrix

Nice	Good	Beautiful	Black
1	0	0	0
0	1	0	0
1	0	1	0
1	0	0	1

Example 02 – Representing Textual Features as N-grams

 N-gram = word

- ▮ Min length of N = 3
- ▮ Max length of N = 3

 Stop Words Removal = True

 Max Features = 4

 Feature method = Count or (Term Frequency Count) Weight

 Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 02 – Representing Textual Features as N-grams



Document-Term Matrix

Beautiful Car Nice	Black Car Car	Car Nice Car	Nice Black Car
0	0	0	0
0	0	0	0
1	1	1	0
0	1	0	1

Example 03 – Representing Textual Features as N-grams

 N-gram = word

- ▣ Min length of N = 1
- ▣ Max length of N = 3

 Stop Words Removal = True

 Max Features = 10

 Feature Method = Count or (Term Frequency Count) Weight

 Feature Extraction - Corpus

Comment	class
a nice car	positive
a good car	positive
a beautiful car a nice car	positive
a nice black car car	positive

Example 03 – Representing Textual Features as N-grams



Document-Term Matrix

Beautiful	Beautiful Car	Beautiful Car Nice	Black	Black Car	Black Car Car	Car	Car Car	Nice	Nice Car
0	0	0	0	0	0	1	0	1	1
0	0	0	0	0	0	1	0	0	0
1	1	1	0	0	0	2	0	1	1
0	1	0	1	1	1	2	1	1	0